AEROTRACK: COMPREHENSIVE ARCHITECTURE & ENGINEERING MANUAL

Real-Time Atmospheric Intelligence & On-Demand XGBoost ML Forecasting

End-to-End Technical Breakdown, Mathematics, Architecture, Commands, 31-Commit Evolution & Code Integration Directory

Author / Engineer	Aniket Mohanty (Reg: 25BCE5816)
Repository	github.com/aniketmohanty04/aerotrack
Live Application	aerotrack-three.vercel.app
Live Backend API	aerotrack-backend-1tlt.onrender.com
Tech Stack	FastAPI, XGBoost, Scikit-Learn, React 18, TypeScript, Vite, Tailwind CSS, Leaflet, Recharts
Documentation Target	Senior Technical Review, Commit Evolution Analysis, and Full Systems Manual

Executive Summary

AeroTrack is a full-stack, production-grade environmental intelligence platform that delivers real-time atmospheric telemetry and on-demand 24-hour machine learning forecasts for any geographical coordinate on Earth. By synthesizing global atmospheric reanalysis models (CAMS), numerical meteorological predictions (ECMWF/GFS), and localized autoregressive gradient boosted trees (XGBoost), AeroTrack eliminates reliance on expensive commercial APIs while outperforming traditional persistence baselines by up to 48% in predictive accuracy.

This engineering manual provides an exhaustive, line-by-line post-mortem and system specification. It teaches every mathematical formula, architecture diagram, network data flow, concurrency pattern, caching algorithm, terminal command, commit evolution step, and frontend-backend integration mechanism implemented throughout the project.

1. High-Level System Architecture & Component Interaction

The AeroTrack system follows a modern decoupled client-server architecture designed for zero cold-start latency, fault tolerance, and sub-second rendering across global desktop and mobile devices.

Layer	Technologies	Key Engineering Role
Client Layer (Frontend)	React 18, TypeScript, Vite, Tailwind CSS, Leaflet, Recharts	SPA hosted on Vercel Edge Network. Handles geospatial mapping, responsive charts, autocomplete search, and state synchronization.
API Layer (Backend Gateway)	FastAPI, Starlette, Uvicorn, Python 3.10+, HTTPX	Asynchronous ASGI server hosted on Render Cloud. Exposes 8 RESTful endpoints, manages CORS, normalizes coordinates, and routes requests.
Data & Cache Layer	In-Memory LRU Caches, In-Flight Task Deduplication	Two-tier caching: 10-min shared atmospheric cache, 30-min ML artifact cache. Collapses thundering herd requests via Promise pooling.
Machine Learning Engine	XGBoost Regressor, Scikit-Learn, Pandas, NumPy, ThreadPool	Trains on-demand in <1.2s on 92 historical days of localized observations. Implements autoregressive recursive multi-step forecasting.
Uncertainty Quantification	Split Conformal Prediction, Non-Parametric Residuals	Computes distribution-free 90% confidence prediction intervals, dynamically expanding across the 24-hour horizon.
Upstream Data Providers	Open-Meteo (CAMS & Weather), Nominatim OSM, BigDataCloud	Provides criteria atmospheric gases, numerical weather reanalysis, text autocomplete, and two-tier reverse geocoding.

End-to-End Query Lifecycle & Execution Sequence

1. **Geospatial Selection:** The client captures latitude and longitude coordinates, updating UI state immediately.
2. **Reverse Geocoding:** The client queries `/api/reverse-geocode` to resolve human-readable administrative location names.
3. **Parallel Async Dispatch:** `App.tsx` fires three independent Axios network requests simultaneously using an `AbortController` signal: telemetry (`/api/air-quality`), historical trends (`/api/trends`), and predictive forecast (`/api/predict`).
4. **Coordinate Normalization & Keying:** The backend normalizes coordinates to exactly 4 decimal places (`round(lat, 4), round(lon, 4)`), giving ~11.1m precision and preventing cache key collisions.
5. **Cache & Task Deduplication:** The backend inspects `SHARED_AIR_CACHE`. If fresh, it returns immediately. If an upstream fetch is already running for those coordinates, it awaits the existing promise rather than firing a duplicate request.
6. **Progressive UI Rendering:** Live telemetry arrives in ~200ms, immediately populating the primary AQI card and criteria pollutant grid. Historical trends render at ~350ms. The heavier ML forecast resolves at ~1.2s, smoothly streaming into the predictive forecast chart.

2. Complete 31-Commit Chronological Evolution & Engineering Post-Mortem

Every commit in the AeroTrack repository represents a specific engineering decision, bug mitigation, or performance optimization. Below is the complete chronological history explaining what each commit changed, why it was necessary, and the technical mechanisms involved.

Commit & Title	Problem / Trigger	Engineering Solution & Mechanism
2f006cd Initial commit	Empty repository.	Scaffolded full-stack architecture: React 18 frontend with Vite, and FastAPI backend with uvicorn.
99c707d Move render.yaml	Render Cloud couldn't detect deployment Blueprint inside subfolder.	Relocated render.yaml to repo root so Render infrastructure-as-code parser auto-configures Python environment.
2f39c76 Add prod env config	Frontend hardcoded localhost:8000; broke on Vercel.	Created frontend/.env.production setting VITE_API_URL to production Render backend, resolved at build time.
abdc979 & b286544 README Unicode fixes	Raw LaTeX ($...$) broke on mobile and GitHub markdown views.	Replaced LaTeX with standard Unicode symbols (µg/m³, —, ·) for clean universal documentation rendering.
bcaffb8 UptimeRobot note sync	Documentation inconsistencies regarding Render free tier sleep.	Standardized documentation explaining 5-min UptimeRobot ping strategy to keep backend container warm 24/7.
19736d6 Reverse-geocode redirects	Nominatim 301/302 redirects caused sporadic geocoding failures.	Configured HTTPX client with follow_redirects=True and added robust fallback error parsing.
68cd0ca Auto-open map popup	User clicked map but perceived UI lag waiting for network response.	Leaflet popup now opens immediately in 'Calculating...' state on click, resolving asynchronously when telemetry arrives.
4075033 & 7dd1bf0 English names & regex	Foreign locations returned non-Latin scripts (Devanagari, Kanji).	Enforced 'accept-language: en' in Nominatim headers; added regex cleansing to sanitize administrative labels.
ef13ea8 Refactor ML pipeline	Missing data caused earlier code to generate fake synthetic waves.	Eliminated deceptive synthetic fallbacks; enforced strict data provenance with explicit error models and tests.
ab46fdb Historical trend pipeline	Open-Meteo payload mixed past days and future forecasts together.	Strictly separated true observations at current.time; normalized UTC timestamps and computed pure 24h rolling window.
748bc04 Reproducible offline eval	Model accuracy unproven against persistence; target leakage risks.	Created backend/evaluate_model.py; benchmarked XGBoost vs persistence across 5 climate zones (proven 32-48% lower MAE).
222a3f9 Split conformal intervals	Gaussian ±1.96σ heuristic assumes normal errors, failing on skewed AQI.	Implemented Split Conformal Prediction using past 168h calibration residuals, providing 90% non-parametric coverage.
4ecd729 Vectorize rollout loops	Recursive autoregressive loop took ~3.5s due to DataFrame appends.	Vectorized rollout feature construction with NumPy arrays and added calibration caching; dropped latency to <1.2s.
84833df & 8a976db Evaluation dataset fixes	Evaluation test set inadvertently included future provider forecasts.	Excluded future forecasts from evaluation test sets; added timedelta math and committed verified EVALUATION_REPORT.md.
f46b7fa Clear stale AQI on click	Selecting a new city briefly displayed the old city's AQI score.	Added instant state reset in App.tsx: clears previous telemetry immediately upon new coordinate selection.
c96f721 Reverse geocode cache	Rapid clicks caused excessive external Nominatim API calls.	Added 24h reverse geocode cache and IN_FLIGHT_AIR_REQUESTS task deduplication to collapse duplicate in-flight requests.

2c4c3d7 Click race condition fix	Slower async response from older click overwrote newer click results.	Implemented monotonic reqIdRef counter and AbortController in App.tsx; stale asynchronous arrivals are discarded.
9497b8c Bundle frontend assets	Production assets needed compilation.	Ran Vite production build generating hashed JS/CSS assets into dist/.
78e4d0b & acadc7b Decouple static mounts	FastAPI had legacy static mounts interfering with Pure API routing.	Removed static file mounts; converted root route (/) to pure JSON manifest pointing to live Vercel frontend.
98d9f73 Remove duplicate health	Duplicate /api/health route declaration in backend/main.py.	Cleaned duplicate route and added test_manifest tests validating all 8 API endpoints.
e27cfa9 EPA 35.5 & dynamic weather	Caution line labeled 35.0 instead of official EPA 35.5 breakpoint.	Corrected EPA breakpoint strictly to 35.5 µg/m³; dynamically injected forecasted weather variables into ML rollout.
ee4ce39 Multi-tier reverse geocode	Nominatim 429 rate-limiting broke location name resolution.	Added BigDataCloud as automated secondary geocode fallback; implemented causal forward-fill imputation.
d2b5bcd Non-blocking ML threads	Heavy XGBoost CPU matrix training froze FastAPI's async event loop.	Offloaded ML training to concurrent.futures.ThreadPool Executor(max_workers=4); event loop kept sub-5ms responsive.
a372f1f Ignore report files	Local analysis reports accidentally staged into Git.	Configured .gitignore to strictly exclude *.html, *.pdf, and *.pptx files.
681146c Clamp AQI 500 & strict TTL	Open-Meteo returned 538, 1055 AQI; cache keys held expired entries.	Clamped us_aqi and european_aqi at 500; added is_capped flag; added MAX_CACHE_SIZE=500 LRU eviction and clear-cache route.
a581ebf AQI capped badge in UI	Users needed visual indication when AQI hit the EPA scale ceiling.	Added amber 'EPA Scale Max (Capped at 500)' badge in PollutantCards.tsx whenever is_capped is true.
4eafc4b Explicit sensor timestamp	Users perceived AQI as stuck because worst-pollutant bracket stayed 156.	Added 'Data from: [Time]' badge in PollutantCards.tsx exposing exact Open-Meteo update timestamp directly on screen.

3. US EPA AQI Mathematics & The Famous >500 Bug

The Mathematical Formulation

The United States Environmental Protection Agency (US EPA) defines the Air Quality Index (AQI) as an arbitrary piecewise linear function mapping physical pollutant mass concentrations into a standardized index ranging from 0 to 500. The mathematical formula governing each individual pollutant sub-index is:

```
I_high - I_low
I_p = -------------------- * (C_p - C_low) + I_low
C_high - C_low

Where:
• I_p = Calculated Air Quality Index for pollutant p
• C_p = Truncated/rounded raw physical concentration of pollutant p
• C_low = Concentration breakpoint that is <= C_p
• C_high = Concentration breakpoint that is >= C_p
• I_low = Index breakpoint corresponding to C_low
• I_high = Index breakpoint corresponding to C_high
```

Overall Composite AQI Principle: The composite Air Quality Index for any location is governed strictly by the *maximum operator* across all monitored criteria pollutants: `Overall_AQI = max(AQI_PM2.5, AQI_PM10, AQI_O3, AQI_NO2, AQI_SO2, AQI_CO)`. Under atmospheric physics, air quality is only as safe as the single worst pollutant present.

The Famous >500 Bug & Root-Cause Post-Mortem

During live testing in severe atmospheric episodes (e.g. winter temperature inversions in Delhi and Gurgaon), the upstream Open-Meteo API returned raw AQI values of 538, 1055, and even 1769. These values alarmed users and broke standard AQI visualization scales.

Root Cause: The official US EPA standard strictly terminates at an AQI of 500 (the ceiling of the Hazardous tier). Values above 500 do not exist in the scientific EPA definition. Open-Meteo's upstream data pipeline took extreme physical PM10 concentrations (e.g. 596 µg/m³) and linearly extrapolated the slope past the final table entry rather than clamping at the table boundary.

The Engineering Solution: In `backend/main.py`, clamping logic was implemented immediately upon upstream data reception: `us_aqi = min(500, us_aqi)`, accompanied by `is_capped = True`. On the frontend, an amber badge 'EPA Scale Max (Capped at 500)' is rendered, preserving mathematical compliance.

4. Machine Learning Engine: XGBoost & Split Conformal Intervals

Model Architecture & Feature Engineering Matrix

The forecasting engine (`backend/ml_model.py`) utilizes an `xgboost.XGBRegressor` with histogram binning (`tree_method='hist'`). Training features are dynamically extracted from 92 historical days (~2,208 hourly observations):

1. **Autoregressive Lags:** Captures atmospheric inertia using historical PM2.5 values: lags at 1h, 2h, 3h, 6h, 12h, 24h, and 48h.
2. **Rolling Statistical Windows:** Computes rolling mean and standard deviation over 6-hour, 12-hour, and 24-hour backward windows.
3. **Cyclical Diurnal & Weekly Trigs:** Projects time onto a circle: `sin_hour = sin(2*pi*h/24)`, `cos_hour = cos(2*pi*h/24)`, `sin_day = sin(2*pi*d/7)`, and `cos_day = cos(2*pi*d/7)`.
4. **Coupled Meteorological Variables:** Binds 2m Temperature (°C), 2m Relative Humidity (%), and 10m Surface Wind Speed (km/h) as direct atmospheric predictors.

Autoregressive Recursive Multi-Step Forecasting Loop

Forecasting 24 hours into the future using a single model requires an **iterative autoregressive loop**. The model predicts hour t+1, appends the prediction into the feature lag buffer, recalculates rolling statistics, injects the *true forecasted weather variables* for hour t+2, and predicts hour t+2 recursively up to step 24. Physical realism is strictly enforced by bounding predictions: `y_pred = max(0.0, y_pred)`.

Split Conformal Prediction (90% Confidence Envelope)

AeroTrack implements **Split Conformal Prediction**, providing a mathematically guaranteed, distribution-free coverage bound:

- **Calibration Holdout:** The past 168 hours (7 days) of observations are held out as calibration set D_cal.
- **Absolute Residuals:** Non-conformity scores are computed: `R_i = |y_i - y_pred_i|`.
- **Conformal Quantile:** For target coverage 1 - alpha = 0.90, the conformal quantile is computed as: `q_hat = Quantile(R, ceil((n+1)*0.90) / n)`.
- **Expanding Horizon Uncertainty:** Forecast error compounds with time. AeroTrack horizontally expands the band across the 24-hour horizon: `half_width_t = q_hat * sqrt(1 + 0.04 * step_idx)`.
- **Clamped Bounds:** `lower_t = max(0.0, y_pred_t - half_width_t)` and `upper_t = y_pred_t + half_width_t`.

5. Frontend Architecture & Client-Server Integration

Request Cancellation & Race Condition Protection

A common flaw in geospatial web applications occurs when a user rapidly clicks Location A and then Location B. Because Location A's slower ML forecast may take 1.2s to resolve while Location B's telemetry resolves in 200ms, Location A's forecast could resolve *after* Location B, overwriting the UI with stale, mismatched data.

AeroTrack eliminates this race condition via two guards: 1. `abortControllerRef.current.abort()` cancels all in-flight HTTP requests immediately upon a new click. 2. A monotonic request ID `reqIdRef.current` discards any asynchronous arrivals whose ID does not match the active query.

6. Complete Terminal Commands Handbook

```
# --- BACKEND COMMANDS (PowerShell / Windows) ---
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
pytest tests/ -v
python evaluate_model.py

# --- FRONTEND COMMANDS ---
cd frontend
npm install
npm run dev
npm run build

# --- VERIFICATION CURLS ---
curl -s https://aerotrack-backend-1tlt.onrender.com/api/health
curl -s https://aerotrack-backend-1tlt.onrender.com/api/air-quality/28.4595/77.0266
```

7. Code Integration Directory: What, Where & Why

File Location	Key Functions	What It Does & Why It Was Used
backend/main.py	get_air_quality() predict_pm25() get_aqi_category()	**WHAT:** FastAPI gateway. **WHY:** Coordinates 8 routes, 2-tier caching, 500 AQI cap, and ThreadPool offloading.
backend/ml_model.py	train_and_forecast_pm25() _recursive_forecast_loop()	**WHAT:** ML engine. **WHY:** Builds lag matrices, cyclical features, trains XGBoost, dynamically injects future weather, and computes 90% conformal intervals.
backend/evaluate_model.py	evaluate_city_forecast()	**WHAT:** Evaluation framework. **WHY:** Benchmarks XGBoost against persistence across 5 climate zones to prove accuracy.
frontend/src/App.tsx	fetchAllData()	**WHAT:** Root React component. **WHY:** Coordinates Axios requests, AbortController cancellation, and layout.
frontend/src/components/AirMap.tsx	AirMap component	**WHAT:** Leaflet interactive map. **WHY:** Cartography, GPS pin location, and click-to-query reverse geocoding.
frontend/src/components/PollutantCards.tsx	PollutantCards component	**WHAT:** Telemetry cards. **WHY:** Displays 6 criteria gases with EPA color coding, overall AQI score, capped 500 badge, and freshness timestamp.

frontend/src/components/ ForecastChart.tsx	ForecastChart component	**WHAT:** 24h predictive chart. **WHY:** Renders hourly trajectory with shaded Split Conformal 90% uncertainty band.

End of AeroTrack Comprehensive Engineering Manual — Built with Pride by Aniket Mohanty